

Hyperparameter Optimization

Chen Hajaj

Why?

Models are composed of hyperparameters.

Hyperparameter:

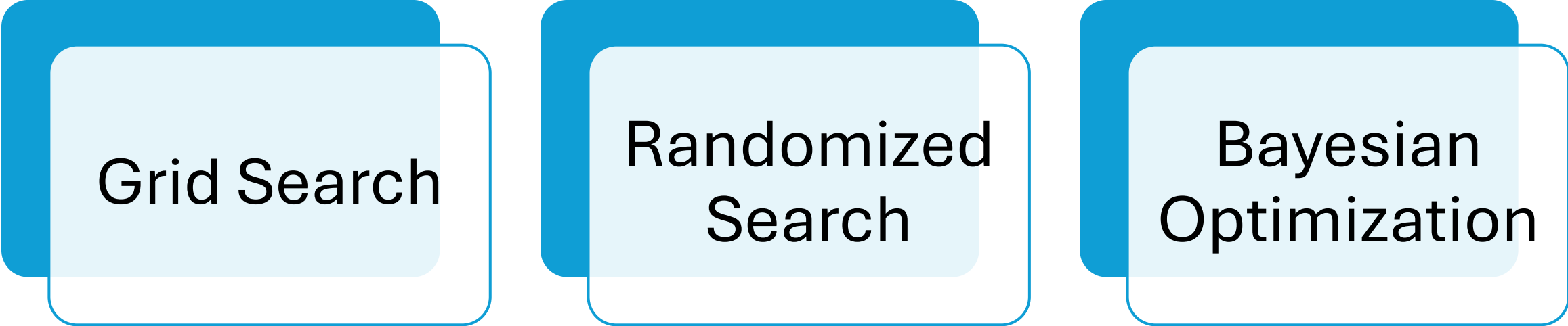
- A **model configuration** set by the developer *before* training.
- Guides how the model learns from a specific dataset.

Important:

- **Hyperparameters are NOT parameters!**
- **Parameters** are learned automatically during training (e.g., weights in a linear regression model).



Three techniques



Grid Search

Randomized
Search

Bayesian
Optimization

Grid Search

- Spot-checking combinations manually:
 - Tedious and time-consuming.
 - Often impractical for large models or datasets.
- Solution:
 - Use Scikit-Learn's GridSearchCV to automate the process!
 - You provide:
 - Parameters to optimize.
 - Possible values for each parameter.

Randomized Search

- What happens when you want to optimize many parameters?
- Each parameter can have many possible optimal values
- Use RandomizedSearch for combinations that you would not have guessed intuitively

Don't forget the CV??

- [RandomizedSearchCV](#) for random search and [GridSearchCV](#) for grid search
- Both techniques use Cross-Validation
- Trains models for different combinations.
- Evaluates them using cross-validation.
- Returns the best combination based on performance.

```
cv = RepeatedStratifiedKFold(n_splits=10, n_repeats=3,  
random_state=1)
```



Scoring	Function	Comment
Classification		
'accuracy'	metrics.accuracy_score	
'balanced_accuracy'	metrics.balanced_accuracy_score	
'average_precision'	metrics.average_precision_score	
'neg_brier_score'	metrics.brier_score_loss	
'f1'	metrics.f1_score	for binary targets
'f1_micro'	metrics.f1_score	micro-averaged
'f1_macro'	metrics.f1_score	macro-averaged
'f1_weighted'	metrics.f1_score	weighted average
'f1_samples'	metrics.f1_score	by multilabel sample
'neg_log_loss'	metrics.log_loss	requires predict_proba support
'precision' etc.	metrics.precision_score	suffixes apply as with 'f1'
'recall' etc.	metrics.recall_score	suffixes apply as with 'f1'
'jaccard' etc.	metrics.jaccard_score	suffixes apply as with 'f1'
'roc_auc'	metrics.roc_auc_score	
'roc_auc_ovr'	metrics.roc_auc_score	
'roc_auc_ovo'	metrics.roc_auc_score	
'roc_auc_ovr_weighted'	metrics.roc_auc_score	
'roc_auc_ovo_weighted'	metrics.roc_auc_score	
Regression		
'explained_variance'	metrics.explained_variance_score	
'max_error'	metrics.max_error	
'neg_mean_absolute_error'	metrics.mean_absolute_error	
'neg_mean_squared_error'	metrics.mean_squared_error	
'neg_root_mean_squared_error'	metrics.mean_squared_error	
'neg_mean_squared_log_error'	metrics.mean_squared_log_error	
'neg_median_absolute_error'	metrics.median_absolute_error	
'r2'	metrics.r2_score	
'neg_mean_poisson_deviance'	metrics.mean_poisson_deviance	
'neg_mean_gamma_deviance'	metrics.mean_gamma_deviance	


```

1  import numpy as np
2  from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
3  from sklearn.linear_model import LinearRegression
4  from sklearn.datasets import make_regression
5
6  # Generate sample data
7  X, y = make_regression(n_samples=100, n_features=5, random_state=42)
8
9  # Initialize the model
10 lr = LinearRegression()
11
12 # Define parameter grid for Grid Search
13 param_grid = {
14     'fit_intercept': [True, False],
15     'normalize': [True, False],
16 }
17
18 # Grid Search
19 grid_search = GridSearchCV(
20     estimator=lr,
21     param_grid=param_grid,
22     cv=5,
23     scoring='neg_mean_squared_error',
24     n_jobs=-1
25 )
26
27 # Fit Grid Search
28 grid_search.fit(X, y)
29
30 print("Grid Search Best Parameters:", grid_search.best_params_)
31 print("Grid Search Best Score:", -grid_search.best_score_) # Negative because of scoring metric
32
33 # Random Search
34 random_search = RandomizedSearchCV(
35     estimator=lr,
36     param_distributions=param_grid,
37     n_iter=5,
38     cv=5,
39     scoring='neg_mean_squared_error',
40     n_jobs=-1,
41     random_state=42
42 )
43
44 # Fit Random Search
45 random_search.fit(X, y)
46
47 print("\nRandom Search Best Parameters:", random_search.best_params_)
48 print("Random Search Best Score:", -random_search.best_score_)
49

```

Code example

- Inputs:
 - Data
 - Model
 - Search space
- Output:
 - Best params
 - Score

Bayesian Optimization

- When a region of hyperparameters turns out to be **good**, it should be **explored more** (instead of trying all possibilities equally).
- Scikit-Optimize (Skopt) <https://scikit-optimize.github.io/>
- TL;DR: pip install scikit-optimize

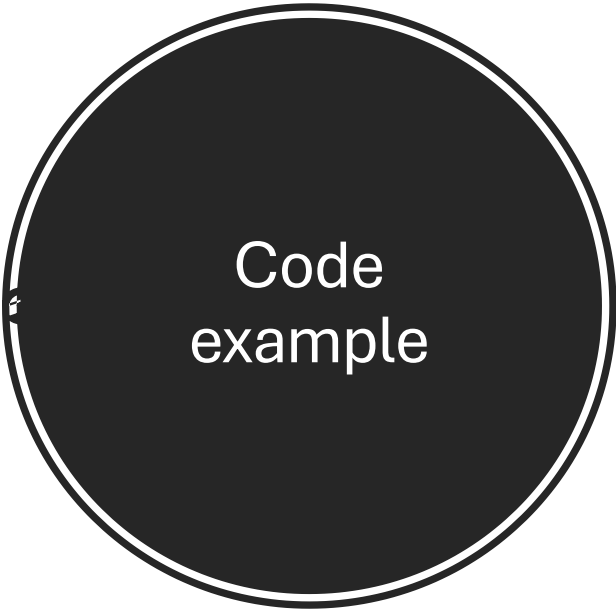
Bayesian optimization with skopt

We are interested in solving

$$x^* = \underset{x}{\operatorname{argmin}} f(x)$$

under the constraints that:

- f is a black box for which no closed form is known (nor its gradients);
- f is expensive to evaluate;
- and evaluations of $y=f(x)$ may be noisy.



Code example

```
1 from skopt import BayesSearchCV
2 from skopt.space import Integer
3
4 # Define the search space for hyperparameters
5 search_space = {
6     "max_depth": Integer(6, 20) # max_depth values from 6 to 20
7 }
8
9 # Set up the Bayesian search with cross-validation
10 forest_bayes_search = BayesSearchCV(
11     estimator=forest_clf,
12     search_spaces=search_space,
13     n_iter=32, # Number of parameter settings sampled
14     scoring="accuracy", # Evaluation metric
15     n_jobs=-1, # Use all available cores
16     cv=5 # 5-fold cross-validation
17 )
18
19 # Fit the search to the training data
20 forest_bayes_search.fit(X_train, y_train)
21
```